

8.API ORM (II)

SISTEMAS DE GESTIÓN EMPRESARIAL
2ºDAM

Bibliografía

[Odoo 11 Development Essentials - Third Edition](#)

ISBN-13: 978-1788477796

[Odoo 11 Development Cookbook](#)

ISBN-13: 978-1788471817

[Working with Odoo 11 - Third Edition: Configure, manage, and customize your Odoo system](#)

ISBN-13: 978-1788476959

Índice

- 1. Métodos ORM: Create/update**
- 2. Métodos ORM: Search/Read**
- 3. Unlink**

Métodos ORM

- **search(cr, uid, [('param1', OP, value)])**

SELECT id FROM table WHERE param1 OP value

- **create(cr, uid, {'param1':value})**

INSERT INTO table VALUES param1

- **write(cr, uid, [ids], {'param1':value})**

UPDATE table SET param1 = value WHERE id = id

- **unlink(cr, uid, [ids])**

DELETE FROM table WHERE id = id

API ORM: create()

- El método create() toma un diccionario para asignar valores a los distintos campos y devuelve el registro creado.
- Los valores por defecto se aplican automáticamente.
- En el siguiente ejemplo se crean tres registros:
 - Uno para la empresa principal asociada (partner_val).
 - Uno para cada contacto, que están disponibles en *record.child_ids*.

API ORM: create()

```
today_str = fields.Date.context_today()
val1 = {'name': 'Eric Idle', 'email': 'eric.idle@example.com', 'date': today_str}
val2 = {'name': 'John Cleese', 'email': 'john.cleese@example.com', 'date': today_str}
partner_val = {'name': 'Flying Circus',
               'email': 'm.python@example.com',
               'date': today_str,
               'is_company': True,
               'child_ids': [(0, 0, val1),
                             (0, 0, val2),] }
record = self.env['res.partner'].create(partner_val)
```

API ORM: create()

- Los valores de los campos de texto se proporcionarán como cadenas de Python (preferiblemente cadenas Unicode).
- Los valores de campos de los tipos Float e Integer se proporcionarán utilizando número reales o enteros de Python.
- Para los campos booleanos se deben utilizar booleanos de Python o números enteros.
- Los valores de los campos de tipo fecha (Date) o fecha y hora (Datetime) se deben proporcionar como cadenas de Python.

API ORM: create()

- En este ejemplo se convierte un campo fecha de Odoo a formato cadena, se le suma una hora y se vuelve a guardar en un campo fecha de Odoo:

```
@api.onchange('date_begin')
def onchange_date_begin(self):
    if self.date_begin and not self.date_end:
        date_begin = fields.Datetime.from_string(self.date_begin)
        self.date_end = fields.Datetime.to_string(date_begin + timedelta(hours=1))
```


API ORM: create()

- Los valores de campos binarios se pasarán como una cadena codificada en Base64.
- Los valores de campo Many2one se proporcionan con un número entero, que debe ser el ID del registro relacionado.
- Los campos One2many y Many2many usan una sintaxis especial. El valor a proporcionar consiste en una lista que contiene tuplas de tres elementos que pueden tener los dos siguientes formatos:

```
# Registro nuevo a partir de los valores del diccionario  
(0 , 0 , { 'c1': v1, 'c2': v2})
```

```
# Relacionar con registros ya existentes  
# Lista de ID's de los registros a relacionar  
# Cuando se utiliza en una relación One2many  
# se eliminará cualquier relación previa de esos registros  
(6, 0, [7, 8, 10])
```

API ORM: update()

Existen tres formas de actualizar un registro en la base de datos.

API ORM: update()

- Método 1: Asignar un valor a un campo en un recordset 'singleton'

```
@api.model
def add_contacts(self, partner, contacts):
    partner.ensure_one()
    if contacts:
        partner.date = fields.Date.context_today()
        partner.child_ids |= contacts
```

API ORM: update ()

- Método 2: Usar el método update () pasando los valores a actualizar mediante un diccionario. Solo funciona para conjuntos de registros de longitud 1 ('singleton').

```
@api.model
def add_contacts(self, partner, contacts):
    partner.ensure_one()
    if contacts:
        today = fields.Date.context_today()
        partner.update({'date': today,
                        'child_ids': partner_child_ids | contacts})
```

API ORM: update() → write()

- Método 3: Usar el método write() pasando los valores a actualizar mediante un diccionario. Este método funciona para conjuntos de registros de tamaño arbitrario y actualizará todos registros con los valores especificados en una sola operación de base de datos cuando las dos anteriores las opciones realizan una llamada a la base de datos por registro y por campo.
- Utiliza un formato especial para el tratamiento de campos relacionales

Tuple	Effect
(0, 0, dict_val)	This creates a new record that will be related to the main record.
(1, id, dict_val)	This updates the related record with the specified ID with the supplied values.
(2, id)	This removes the record with the specified ID from the related records and deletes it from the database.
(3, id)	This removes the record with the specified ID from the related records. The record is not deleted from the database.
(4, id)	This adds an existing record with the supplied ID to the list of related records.
(5,)	This removes all the related records, equivalent to calling (3, id) for each related id.
(6, 0, id_list)	This creates a relation between the record being updated and the existing record, whose IDs are in the Python list id_list.

API ORM: search()

- Toma una expresión de dominio (filtro de registros) y devuelve un conjunto de registros (recordset) coincidentes.
- Este conjunto de registros puede seleccionarse utilizando varios parámetros:
 - *offset*: número de registro a partir del cual se recuperan los registros del conjunto
 - *limit*: número máximo de registros a recuperar en una ejecución de search
 - *order*: especifica el campo por el que se ordena el resultado

```
# search en el modelo actual: res.partner
resultado = self.search([('is_company', '=', True), ('customer', '=', True)])
# resultado ---> res.partner(7, 18, 12, 14)
print (self.search([('is_company', '=', True)], limit=1).name)
# en pantalla --> 'Agrolait'
```

API ORM: unlink ()

- El método *unlink()* permite borrar un registro o un conjunto de registros de la base de datos.
- Si lo que queremos es borrar registros de una relación se debe utilizar el método *write()* utilizando las tuplas:
 - (2, id) - que elimina la relación y el registro relacionado de la base de datos
 - (3,id) - que elimina la relación
 - (5,) - que elimina todos los registros existentes en la relación

API ORM: browse()

- El método *browse()* toma una lista de ID's de la base de datos y recupera un recordset con dichos registros.

```
recordset_clientes = self.env['res.partner'].browse([7, 18, 12])
```